

Nouvelle Calédonie – 2025 – sujet1 - Correction

Exercice 3 (8 points)

Partie A — Modélisation

1) Pourquoi les cibles sont entre 1 et 45 ?

Chaque ligne contient 5 chiffres compris entre 1 et 9.

- Minimum possible : garder un seul chiffre 1 $\rightarrow$ somme minimale = **1**
- Maximum possible : garder les 5 chiffres 9 $\rightarrow$ somme maximale = **45**

👉 Donc les cibles sont nécessairement entre **1 et 45**.

2) Ligne [6, 4, 5, 8, 2]

- Plus petite cible possible : garder un seul chiffre minimal $\rightarrow$ **2**
 - Plus grande cible possible : $6+4+5+8+2 =$ **25**
-

3) Fonction `extraireLigne`

```
def extraireLigne(plateau, i):  
    return plateau[i]
```

4) Fonction `extraireColonne`

```
def extraireColonne(plateau, i):  
    colonne = []  
    for ligne in plateau:  
        colonne.append(ligne[i])  
    return colonne
```

👉 Version plus concise, en compréhension:

```
def extraireColonne(plateau, i):  
    return [ligne[i] for ligne in plateau]
```

Partie B — Règles de simplification

5) Plateau solution (figure 1)

On remplace les cases supprimées par 0.

Exemple cohérent avec les cibles données :

```
plateau_solution = [  
    [7, 0, 2, 0, 2],  
    [0, 6, 3, 0, 0],  
    [7, 0, 3, 2, 0],  
    [6, 0, 0, 0, 0],  
    [0, 0, 0, 0, 4]  
]
```

6) Règle 1 — appliquer à chaque ligne

Principe : On supprime les nombres strictement supérieurs à la cible de la ligne.

Fonction regle1 (complétée):

```
def regle1(plateau, ciblesLignes, ciblesColonnes):  
  
    # Traitement des lignes  
    for i in range(5):  
        for j in range(5):  
            if plateau[i][j] > ciblesLignes[i]: # ligne 4  
                plateau[i][j] = 0  
  
    # Traitement des colonnes  
    for j in range(5):  
        for i in range(5):  
            if plateau[i][j] > ciblesColonnes[j]: # ligne 10  
                plateau[i][j] = 0
```

7) Fonction unImpair

```
def unImpair(liste):
    compteur = 0
    for x in liste:
        if x % 2 == 1:
            compteur += 1
    return compteur == 1
```

8) Fonction regle2 (complétée)

```
def regle2(plateau, ciblesLignes, ciblesColonnes):

    # Lignes
    for i in range(5):
        if ciblesLignes[i] % 2 == 0 and unImpair(plateau[i]):    # ligne 4
            for j in range(5):
                if plateau[i][j] % 2 == 1:    # ligne 5
                    plateau[i][j] = 0

    # Colonnes
    for j in range(5):
        colonne = extraireColonne(plateau, j)
        if ciblesColonnes[j] % 2 == 0 and unImpair(colonne):    # ligne 11
            for i in range(5):
                if plateau[i][j] % 2 == 1:    # ligne 12
                    plateau[i][j] = 0
```

Partie C — Masques binaires

9) Pourquoi [1,1,0,0,1] est solution ?

Liste : [1,2,3,5,2]
Masque : [1,1,0,0,1]

On garde : $1 + 2 + 2 = 5$

Donc somme = cible.

Autres masques solutions :

- [0,1,1,0,0] $\rightarrow 2+3 = 5$
- [1,0,0,1,0] $\rightarrow 1+5 = 6$ ❌
- [0,0,0,1,0] $\rightarrow 5$

Donc solutions :

```
[1,1,0,0,1]
[0,1,1,0,0]
[0,0,0,1,0]
```

9) Fonction somme

```
def somme(liste, masque):
    total = 0
    for i in range(5):
        if masque[i] == 1:
            total += liste[i]
    return total
```

10) 26 en binaire sur 5 bits

$$26 = 16 + 8 + 2$$

👉 [1, 1, 0, 1, 0]

12) Pourquoi seulement 0 à 31 ?

Sur 5 bits :

$$\text{Maximum} = 2^5 - 1 = 31$$

13) Fonction dec2bin

```
def dec2bin(n):
    bits = []
    for i in range(4, -1, -1):
        bits.append((n // (2**i)) % 2)
    return bits
```

Version plus classique :

```
def dec2bin(n):
    bits = []
    for i in range(5):
        bits.insert(0, n % 2)
        n = n // 2
    return bits
```

14) Fonction masques_solutions

```
def masques_solutions(liste, cible):  
  
    solutions = []  
  
    for n in range(32):  
        masque = dec2bin(n)  
        if somme(liste, masque) == cible:  
            solutions.append(masque)  
  
    return solutions
```

Partie D — Vérification finale

15) Fonction teste_solution

```
def teste_solution(plateau, ciblesLignes, ciblesColonnes):  
  
    # Vérification lignes  
    for i in range(5):  
        if sum(plateau[i]) != ciblesLignes[i]:  
            return False  
        if sum(plateau[i]) == 0:  
            return False    # au moins un chiffre par ligne  
  
    # Vérification colonnes  
    for j in range(5):  
        total = 0  
        for i in range(5):  
            total += plateau[i][j]  
        if total != ciblesColonnes[j]:  
            return False  
  
    return True
```
